

Apex Getaways — Full Documentation

1. What this site is

A travel agency platform: clients search and book flights, hotels, and airport/hotel pickups; pay via Paystack; and track their bookings. Admins manage all bookings, customers, pricing, and content from /admin.

Stack: React + Vite (frontend) · Supabase (auth, database, storage, realtime) · Paystack (payments) · Duffel (real flight search data) · Vercel (hosting + one serverless function).

2. Accounts & Authentication

Roles

Every user has a role in the profiles table: client (default), worker, or admin. There's no separate "admin login" — admins sign in exactly like clients; /admin just checks their role and shows the dashboard instead of "access denied."

Sign up / Sign in

- Email + password, via Supabase Auth.
- If your Supabase project has "Confirm email" turned on (default), new users see a "check your inbox" screen and must click the emailed link before they can log in. Turn this off in Supabase → Authentication → Providers → Email if you want instant login after signup instead.
- Google sign-in is wired up in the code (signInWithGoogle) but not yet exposed as a button on the Login/Register pages.

Forgot password

/auth/forgot-password sends a reset link via Supabase.

Change password (while logged in)

Both roles can change their password without logging out: - **Clients:** Dashboard → Profile tab → "Change Password" section. - **Admin:** /admin/settings → "Change Password" card. Both call the same underlying changePassword() function — there's only one password system, shared by every role.

Becoming an admin

No signup flow makes you an admin automatically (for security). After signing up normally once, run in the Supabase SQL Editor:

```
update public.profiles set role = 'admin' where email = 'you@example.com';
```

Then log out and back in.

3. The booking process — client side

The flow is the same shape for Flights, Hotels, and Pickups:

1. **Search & select** (/booking/flights, /booking/hotels, /booking/pickup)
 - Flights: real search via Duffel (falls back to demo data if Duffel is unreachable or not configured — a banner tells you which one you're seeing).
 - Hotels: currently demo data (see §6).
 - Pickups: guest picks where they're arriving from (country or Nigerian state); price comes from what admin set for that location.
 - Nothing is saved to the database yet — it all lives in BookingContext (backed by sessionStorage, so a page refresh doesn't lose it).
 2. **Extras** — insurance, meal preference, baggage add-ons.
 3. **Passenger details** — names, DOB, passport info, contact info. If logged in, passport scans upload to Supabase Storage and get linked to the passenger record.
 4. **Review** — full summary before paying.
 5. **Payment** — this is the first moment anything hits the database:
 - saveBooking() inserts a row into bookings (status: pending_payment, a generated order ref like APX-XXXXXX, and the entire trip as JSON).
 - Paystack's popup opens for the actual charge.
 - On success, the booking flips to payment_received and stores the Paystack payment reference.
 6. **Confirmation** — client sees their order ref. If logged in, the booking now also appears on their **Dashboard → My Bookings**, live (via Supabase Realtime — no refresh needed when its status changes later).
-

4. The booking process — admin side

/admin/bookings shows every booking, live, with filters by status and type, and a search box (name/email/order ref).

For each booking, admin can: - **Set the PNR** once the airline has actually issued it (see §6 for what "actually issued" means, since Duffel here is data-only). - **Upload the ticket / travel document** — a PDF or image (e-ticket, hotel voucher, itinerary). This uploads to Supabase Storage and becomes the link behind the client's "View Ticket" button on their Dashboard. Re-uploading replaces the previous document. - **Add an admin note** (shown to the client — use it for anything they should know, e.g. "seat request confirmed with airline"). - **Move the status forward**: pending_payment → payment_received → processing → tickets_issued → confirmed (or cancelled / refunded).

Every change is visible to the client immediately, live, on their Dashboard — no refresh needed.

5. Why the admin step matters (data-only flight search)

This is the key thing to understand about how this site works.

Duffel is used purely to *search and price* real flights — nothing here books through Duffel’s order/ticketing endpoints. So when a client “books” a flight on this site, what actually happens is:

1. Client pays through Paystack for the price shown (real Duffel price + your markup).
2. **Admin then goes and actually secures the seat with the airline through your normal agency channel** (your GDS, the airline’s trade portal, or however your agency issues tickets today) — this part is entirely outside the website, same as before any of these API integrations existed.
3. Once the real ticket exists, admin comes back to /admin/bookings, enters the **PNR**, uploads the **e-ticket/document**, and marks the booking **tickets_issued** or **confirmed**.
4. The client sees all of this appear on their Dashboard: PNR, the uploaded document (downloadable), any admin notes, and the updated status.

So the client’s source of truth for “what do I actually have” is always what admin has entered into the booking — the live Duffel search result was only ever used to show an accurate, real-time price at checkout time, not as a live-issued ticket.

If you want this to be fully automatic (client books, ticket appears with no admin step), that requires actually calling Duffel’s Order creation endpoints — a materially bigger, riskier integration (real money movement with the airline, cancellation/refund handling, etc.) — deliberately out of scope here since you said data-only.

6. Data sources — what’s real vs. demo right now

Data	Source	Notes
Flights	Real — Duffel Flights API	Server-side only, via <code>api/flights.js</code> . Falls back to demo data if unreachable.
Hotels	Demo (<code>Math.random()</code>)-generated)	Duffel also has a Stays API (same token) — not yet wired in.
Pickup/Cars	Admin-set fixed prices	Realistic as-is — transfer services are usually a fixed rate card, not “fetched” from anywhere.

7. Pricing & markup (Admin → Settings)

- **Flights & Hotels:** admin sets an exact ₦ **amount** added on top of every fetched price (not a percentage) — e.g. “Flights: ₦15,000” means every flight price shown to clients is the real price + ₦15,000.
 - **Hotel Pickup Pricing:** admin sets an exact price per location — Nigeria as a whole, any individual Nigerian state, or any other country (a searchable dropdown shows each location’s local currency for reference, but the price itself is always entered and shown in Naira). Lookup order: exact state price → whole-Nigeria price → a safe built-in default if nothing’s configured yet.
-

8. Fixed in the last review pass

A full re-check of all three booking flows (Flights, Hotels, Pickup) found and fixed several real pricing/payment bugs: - Duffel flight prices were being charged twice over for multi-passenger bookings (Duffel’s total is for all passengers combined; the app was multiplying by passenger count again on top of that). - Hotel and Pickup bookings couldn’t actually be paid for at all — the fare calculator only ever handled flights, so the total came out as ₦0 and the Pay button was disabled. - Hotel markup was applied inconsistently between the search-results screen and the room-selection screen, and multi-room hotel bookings weren’t charged for all the rooms selected. - Pickup return trips and a stale flight-baggage fee could leak into the wrong booking’s total if a guest abandoned one flow and started another. - The Confirmation page showed no trip details at all for Hotel/Pickup bookings (only worked for flights) — now shows a proper summary for all three.

9. Known gaps / things to decide next

- Hotels are still demo data (Duffel Stays would fix this — same token).
 - Google sign-in isn’t exposed as a button yet, only implemented in code.
 - Only one document per booking can be uploaded (replaces the previous one) — fine for a single e-ticket, but if you need to attach a ticket *and* a separate hotel voucher *and* a visa letter, that needs a small schema change (an array of documents instead of one ticketUrl) — ask if you want this.
 - Tickets bucket is public (anyone with the exact link can view a document) — low risk since links are unguessable UUIDs, but can be tightened to signed URLs later if needed.
-

10. Troubleshooting: “Network error, check your connection” on login

This almost always means the app can’t reach Supabase — usually because `VITE_SUPABASE_URL` / `VITE_SUPABASE_ANON_KEY` aren’t set (or aren’t set correctly) wherever it’s running. Without them, the app silently falls back to a placeholder demo URL that doesn’t exist, and every auth call fails with exactly this message.

Check: 1. Does `.env` (local) or your Vercel project's Environment Variables (deployed) actually have `VITE_SUPABASE_URL` and `VITE_SUPABASE_ANON_KEY` set, with real values from your Supabase project's Settings → API page? 2. If you just added/changed them in Vercel, you need to **redeploy** — env var changes don't apply to already-built deployments. 3. Did you actually run `supabase/schema.sql` in your Supabase project's SQL Editor? (Missing tables cause different errors, but it's worth confirming while you're checking setup.)

See `SUPABASE_SETUP.md` for the full setup walkthrough.